

Utilización avanzada de clases I. Herencia

Esteban Álvarez

Contenidos

- Relaciones entre clases
- Composición
- Herencia

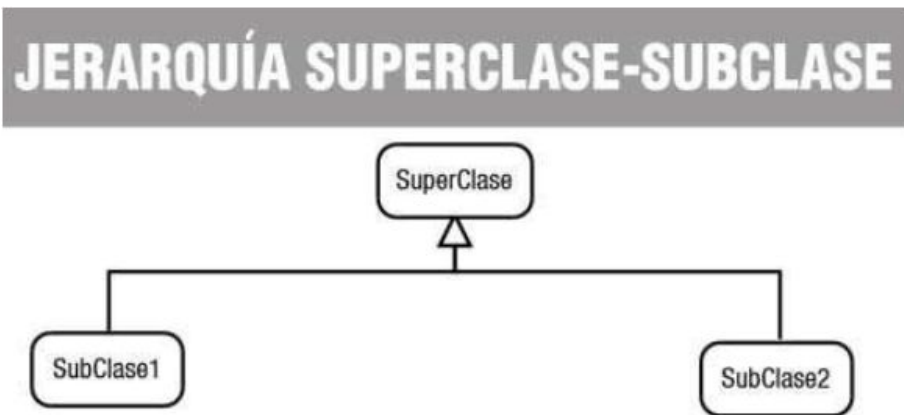
Relaciones entre clases

Se pueden distinguir varios **tipos de relaciones** entre clases:

- **Clientela o Asociación.** Cuando una clase **utiliza objetos** de otra clase. Es el tipo de relación básica que vimos en la Unidad 4
- **Composición.** Cuando alguno de los **atributos** de una clase **es un objeto** de otra clase: un `coche` tiene un atributo `motor` que a su vez es un objeto
- **Anidamiento.** Cuando se definen **clases en el interior de otra clase**, para proteger de una forma más fuerte a dichos objetos
- **Herencia.** Cuando se crea una clase partiendo de una clase o de una jerarquía de clases preexistente, añadiendo alguna funcionalidad específica (**especialización**). `Persona->Alumno` `Mascota->Perro`

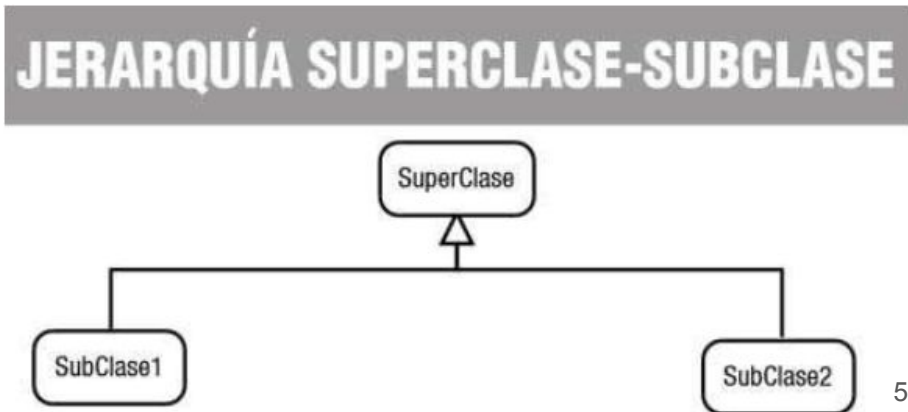
Herencia

- La herencia es el mecanismo que permite crear una clase **a partir de otra que ya existe**
- Las **subclases** heredan los **atributos y métodos** de la **superclase**
- Por tanto evita tener que **reescribir el código** de la clase padre



Herencia

- Nomenclatura
 - La clase **de la que se hereda** puede recibir los nombres:
 - superclase
 - clase padre
 - clase base
 - La clase **que hereda** es la
 - subclase
 - clase hija
 - clase derivada



Herencia

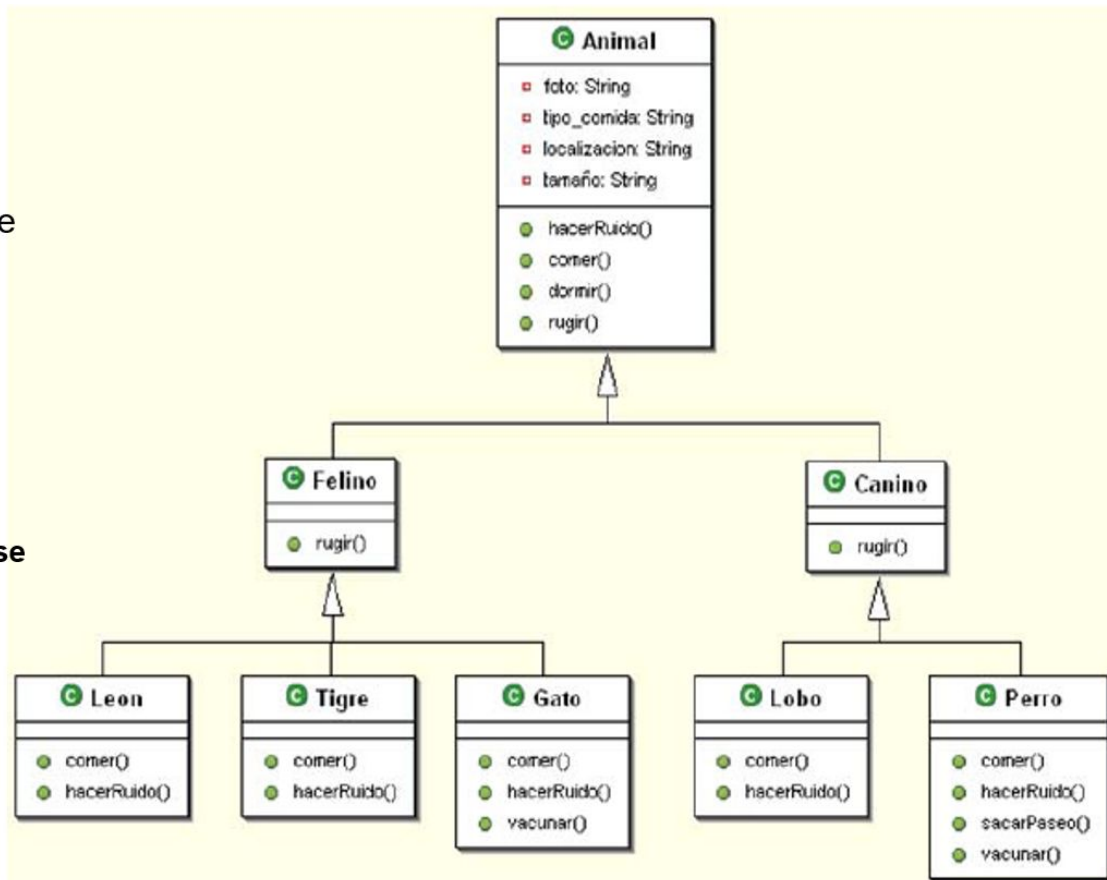
- En la herencia en POO una subclase suele ser una **especificación o especialización** de la superclase:
 - Coche y Moto heredan de Vehículo
 - Un vehículo es algo más genérico que un coche o una moto
 - Coche y Moto heredan todos los atributos y métodos de Vehículo. (Por ejemplo los atributos `marca` y `modelo` y los métodos `parar()` y `arrancar()`). Coche y Moto son Vehículos
 - Además, cada subclase puede especificar sus propios atributos y métodos
 - Coche puede incluir el atributo `numeroDePuertas`
 - No tiene sentido que Moto incluya el atributo anterior pero puede tener el atributo `tipoMotor` que puede tomar los valores “2 tiempos” o “4 tiempos”

Herencia

superclass

superclass / subclass

subclass



Herencia

- La implementación de la herencia se hace mediante la palabra reservada `extends`

```
public class subClase extends superClase{  
  
}
```

```
public class Alumno extends Persona{
```


Herencia

- Ejemplo:

```
public class Coche extends Vehiculo{  
public class Moto extends Vehiculo{
```

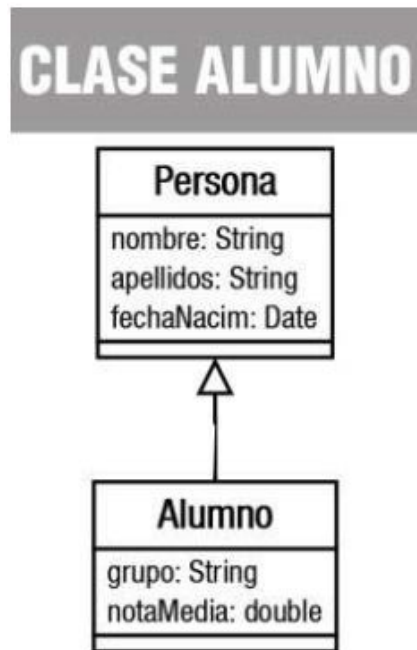
- Coche y Moto heredan de Vehículo

Herencia

- Ejemplo:

```
public class Persona{  
    private String nombre;  
    private String apellidos;  
    private LocalDate fechaNacimiento;  
}
```

```
public class Alumno extends Persona{  
    private String grupo;  
    private double notaMedia;  
}
```



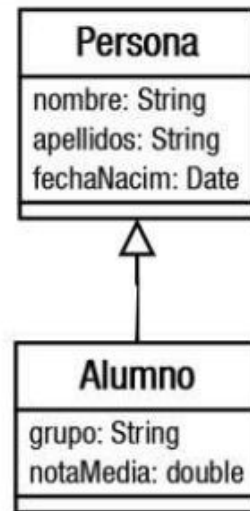
Herencia

En el ejemplo anterior:

- Alumno **hereda** de persona. (Un alumno es una persona)
- Alumno tiene los atributos `grupo` y `notaMedia` propios pero también tiene los **atributos heredados** `nombre`, `apellidos` y `fechaNacimiento`

Lo que se hereda no se compra

CLASE ALUMNO



Herencia

EJERCICIO

- Imagina que también se necesita una clase Profesor, que contará con los atributos nombre, apellidos, fecha de nacimiento, salario y especialidad
- ¿Cómo crearías esa nueva clase y qué atributos le añadirías?
- Implementa las clases Persona, Alumno y Profesor con los atributos especificados y los setters y getters que sean necesarios
- **No** implementes los constructores de Persona, Alumno y Profesor
- **No** implementes el método toString en las clases

```
public class Persona {  
    private String nombre, apellidos;  
    private LocalDate fechaNacimiento;  
  
    public String getNombre() {  
        return nombre;  
    }  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
    public String getApellidos() {  
        return apellidos;  
    }  
    public void setApellidos(String apellidos) {  
        this.apellidos = apellidos;  
    }  
    public LocalDate getFechaNacimiento() {  
        return fechaNacimiento;  
    }  
    public void setFechaNacimiento(LocalDate fechaNacimiento) {  
        this.fechaNacimiento = fechaNacimiento;  
    }  
}
```

```
public class Alumno extends Persona{
    private String grupo;
    private double notaMedia;

    public String getGrupo() {
        return grupo;
    }
    public void setGrupo(String grupo) {
        this.grupo = grupo;
    }
    public double getNotaMedia() {
        return notaMedia;
    }
    public void setNotaMedia(double notaMedia) {
        this.notaMedia = notaMedia;
    }
}
```

```
public class Profesor extends Persona{
    private String especialidad;
    private double salario;

    public String getEspecialidad() {
        return especialidad;
    }
    public void setEspecialidad(String especialidad) {
        this.especialidad = especialidad;
    }
    public double getSalario() {
        return salario;
    }
    public void setSalario(double salario) {
        this.salario = salario;
    }
}
```

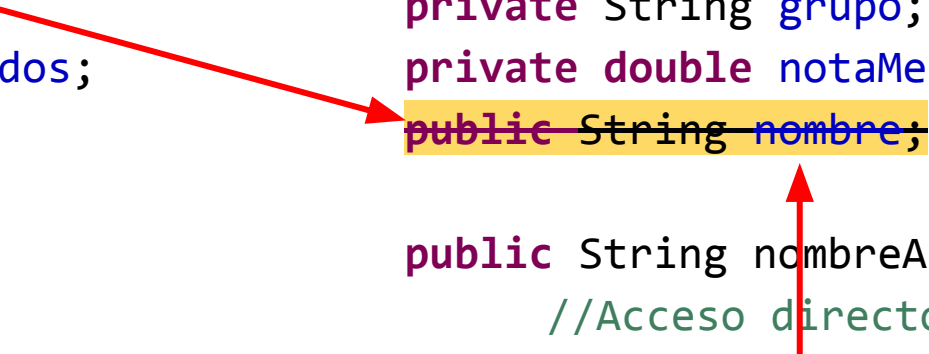
Herencia de atributos

IMPORTANTE

- Cuando los atributos de la superclase son **públicos** (o sin modificador) la subclase tiene **acceso directo** a ellos
- Es como si se “copiasen” esos atributos en la subclase

```
class Persona {  
    public String nombre;  
    private String apellidos;  
}
```

```
class Alumno extends Persona{  
    private String grupo;  
    private double notaMedia;  
    public String nombre;  
  
    public String nombreAlumno() {  
        //Acceso directo a nombre  
        return this.nombre;  
    }  
}
```



Herencia de atributos

IMPORTANTE

- Pero si los atributos de la superclase son **privados** la subclase **NO** tiene **acceso directo** a ellos
- La única forma de acceso es a través de **getters y setters**

```
class Persona {  
    public String nombre;  
    private String apellidos;  
  
    public String getApellidos() {  
        return apellidos;  
    }  
}
```

```
class Alumno extends Persona{  
    private String grupo;  
    private double notaMedia;  
    public String apellidosAlumno() {  
        //Acceso a los apellidos de ESTE objeto  
        //a través del método get (público que se  
        //ha heredado). this es opcional.  
        //No puedo hacer: this.apellidos  
        return this.getApellidos();  
    }  
}
```


Herencia de atributos (y métodos)

CONCLUSIÓN

- Aunque la subclase **hereda todos los atributos y métodos** de la superclase
- solo tiene **acceso** a aquellos que son **públicos** (o sin modificador)
- Para acceder a los **atributos privados heredados** es necesario hacerlo a través de los getters y setters heredados

Herencia de atributos (y métodos)

EJERCICIO

- Asegúrate de que las tres clases anteriores tienen sus atributos privados
- Implementa los métodos `mostrarDatosPersona`, `mostrarDatosAlumno` y `mostrarDatosProfesor` en las clases `Persona`, `Alumno` y `Profesor` respectivamente, teniendo en cuenta que una subclase debe mostrar los valores de sus atributos pero también los heredados de su superclase
- En el main crea un objeto de cada tipo. Asigna valores utilizando los setters y muestra sus datos con `mostrarDatosXXX`

```
//Clase Persona
public String mostrarDatosPersona() {
    return nombre+" "+apellidos+" "+fechaNacimiento;
}
//Clase Alumno
public String mostrarDatosAlumno() {
    return mostrarDatosPersona()+" "+grupo+" "+notaMedia;
}
//Clase Profesor
public String mostrarDatosProfesor() {
    return mostrarDatosPersona()+" "+especialidad+" "+salario;
}
```

```
public static void main(String[] args) {  
    Persona persona = new Persona();  
    Alumno alumno = new Alumno();  
    Profesor profesor = new Profesor();
```

```
NomPersona ApePersona 1990-12-30  
NomAlumno ApeAlumno 2001-01-14 DAM 7.8  
NomProfesor ApeProfesor 1971-09-25 Informática 1000.0
```

```
    persona.setNombre("NomPersona");  
    persona.setApellidos("ApePersona");  
    persona.setFechaNacimiento(LocalDate.of(1990, 12, 30));  
    System.out.println(persona.mostrarDatosPersona());
```

```
    alumno.setNombre("NomAlumno");  
    alumno.setApellidos("ApeAlumno");  
    alumno.setFechaNacimiento(LocalDate.of(2001, 1, 14));  
    alumno.setGrupo("DAM");  
    alumno.setNotaMedia(7.8);  
    System.out.println(alumno.mostrarDatosAlumno());
```

```
    profesor.setNombre("NomProfesor");  
    profesor.setApellidos("ApeProfesor");  
    profesor.setFechaNacimiento(LocalDate.of(1971, 9, 25));  
    profesor.setEspecialidad("Informática");  
    profesor.setSalario(1000);  
    System.out.println(profesor.mostrarDatosProfesor());
```

```
}
```

Constructores

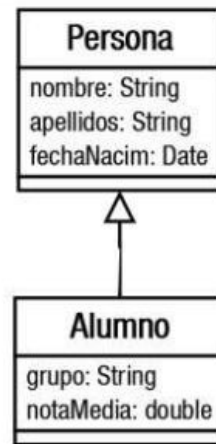
- La superclase y la subclase tendrán **diferentes constructores**
- La **superclase** tiene un constructor que inicializa sus propiedades
- Y el constructor de la **subclase** hace lo propio

- Por tanto, cuando **creamos un objeto de la subclase**, hay que llamar también al constructor de la superclase
- ¿Quién se encarga de hacerlo?

El constructor de la subclase en su primera línea

- Se produce una **llamada en cadena**
- Ejemplo: al crear un objeto “Alumno” llamamos a su constructor y este a su vez, llama al de la clase “Persona”

CLASE ALUMNO



```
class Persona {  
    private String nombre;  
    private String apellidos;  
    private LocalDate fechaNacimiento;  
  
    public Persona() {  
        //Inicializa todos los atributos  
        //a 0, false o null  
    }  
    public Persona(String nombre,  
        String apellidos,  
        LocalDate fechaNacimiento) {  
        this.nombre = nombre;  
        this.apellidos = apellidos;  
        this.fechaNacimiento = fechaNacimiento;  
    }  
}
```

```
class Alumno extends Persona{  
    private String grupo;  
    private double notaMedia;  
  
    public Alumno(String grupo, double notaMedia){  
        //Llamada al constructor sin parámetros  
        //de la superclase  
        super();  
        this.grupo = grupo;  
        this.notaMedia = notaMedia;  
    }  
}
```

```
class Persona {  
    private String nombre;  
    private String apellidos;  
    private LocalDate fechaNacimiento;  
  
    public Persona() {  
        //Inicializa todos los atributos  
        //a 0, false o null  
    }  
    public Persona(String nombre,  
                    String apellidos,  
                    LocalDate fechaNacimiento) {  
        this.nombre = nombre;  
        this.apellidos = apellidos;  
        this.fechaNacimiento = fechaNacimiento;  
    }  
}
```

```
class Alumno extends Persona{  
    private String grupo;  
    private double notaMedia;  
  
    //Constructor. Recibe los parámetros para el  
    //constructor de la clase Alumno y para el de la  
    //clase Persona  
    public Alumno(String nombre, String apellidos,  
                    LocalDate fechaNacimiento,  
                    String grupo, double notaMedia) {  
        //Llamada al constructor de la superclase  
        //con los parámetros que nos pasan  
        super(nombre, apellidos, fechaNacimiento);  
        this.grupo = grupo;  
        this.notaMedia = notaMedia;  
    }  
}
```

```
class Persona {
    private String nombre;
    private String apellidos;
    private LocalDate fechaNacimiento;

    public Persona() {
        //Inicializa todos los atributos
        //a 0, false o null
    }
    public Persona(String nombre,
        String apellidos,
        LocalDate fechaNacimiento) {
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.fechaNacimiento = fechaNacimiento;
    }
}
```

```
class Alumno extends Persona{
    private String grupo;
    private double notaMedia;

    //Un tercer constructor. Inicializa a 0 o null
    //los atributos de Alumno
    public Alumno() {
        //y llama al constructor de la superclase
        //sin parámetros que inicializará los
        //atributos de Persona a 0 o null
        super();
    }
}
```


Constructores

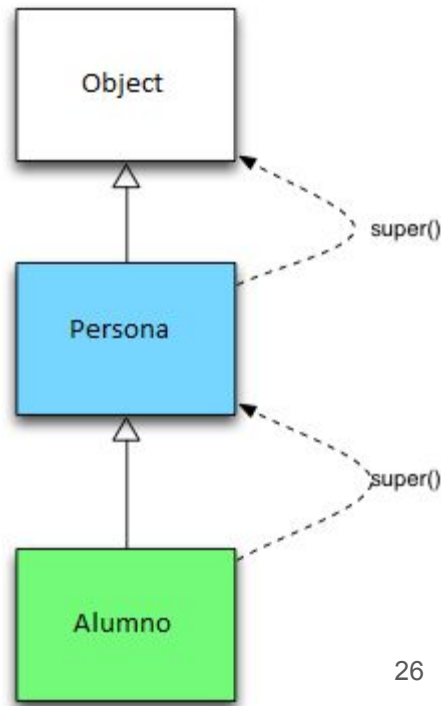
- La llamada al constructor **super()** está implícita aunque no la pongamos en el constructor de la subclase

```
public Alumno() {  
    //Inicializa a 0 o vacío los atributos de Alumno  
    //y llama al constructor de la superclase aunque no  
    //aparece la instrucción super()  
}
```

Constructores

- De hecho la superclase Persona también llama al constructor de su superclase
- Todas las clases **heredan de la clase Object**

```
public Persona() {  
    //Inicializa a 0 o vacío los atributos  
    //de persona y llama al constructor de la  
    //superclase Object  
    //la instrucción super()  
}
```



Constructores

CONCLUSIÓN

- Los constructores **NO SE HEREDAN**
- Las subclases **utilizan** los constructores de las superclases
- La **primera línea** del constructor de una subclase **debe ser** la llamada al constructor de la superclase
- Si se omite se sustituye por **super()**
- Se produce una **llamada de constructores en cadena**

Constructores

EJERCICIO

- Crea constructores con y sin parámetros para cada una de las clases
- Utilízalos en el main y comprueba su funcionamiento con mostrarDatosXXX

```

public Persona() {
    nombre="Sin nombre";
    apellidos="Sin apellidos";
    fechaNacimiento=LocalDate.of(1, 1, 1);
}
public Persona(String nombre, String apellidos, LocalDate fechaNacimiento) {
    this.nombre = nombre;
    this.apellidos = apellidos;
    this.fechaNacimiento = fechaNacimiento;
}

```

```

public Alumno() {
    super(); //Se puede omitir
    grupo="Sin grupo";
    notaMedia=-1;
}
public Alumno(String nombre, String apellidos, LocalDate fechaNacimiento, String grupo, double notaMedia) {
    super(nombre, apellidos, fechaNacimiento);
    this.grupo = grupo;
    this.notaMedia = notaMedia;
}

```

```

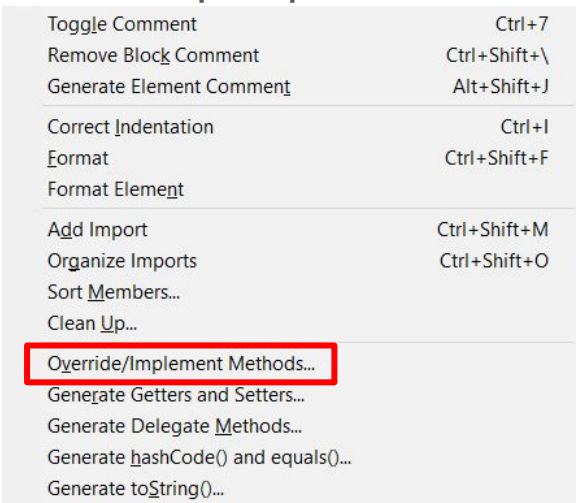
public Profesor() {
    super(); //Se puede omitir
    especialidad="Sin especialidad";
    salario=-1;
}
public Profesor(String nombre, String apellidos, LocalDate fechaNacimiento, String especialidad, double salario) {
    super(nombre, apellidos, fechaNacimiento);
    this.especialidad = especialidad;
    this.salario = salario;
}

```

```
public static void main(String[] args) {  
    Persona persona1 = new Persona();  
    System.out.println("Datos persona 1: " + persona1.mostrarDatosPersona());  
    Persona persona2 = new Persona("NomPers2", "ApePers2", LocalDate.of(1900, 12, 30));  
    System.out.println("Datos persona 2: " + persona2.mostrarDatosPersona() + "\n");  
  
    Alumno alumno1 = new Alumno();  
    System.out.println("Datos alumno 1: " + alumno1.mostrarDatosAlumno());  
    Alumno alumno2 = new Alumno("NomAlum2", "ApeAlum2", LocalDate.of(2002, 10, 21), "1DAM", 7.8);  
    System.out.println("Datos alumno 2: " + alumno2.mostrarDatosAlumno() + "\n");  
  
    Profesor profesor1 = new Profesor();  
    System.out.println("Datos profesor 1: " + profesor1.mostrarDatosProfesor());  
    Profesor profesor2 = new Profesor("NomProf2", "ApeProf2", LocalDate.of(1971, 9, 25), "Informática", 1000);  
    System.out.println("Datos profesor 2: " + profesor2.mostrarDatosProfesor() + "\n");  
}
```

Herencia de métodos. Sobrescritura

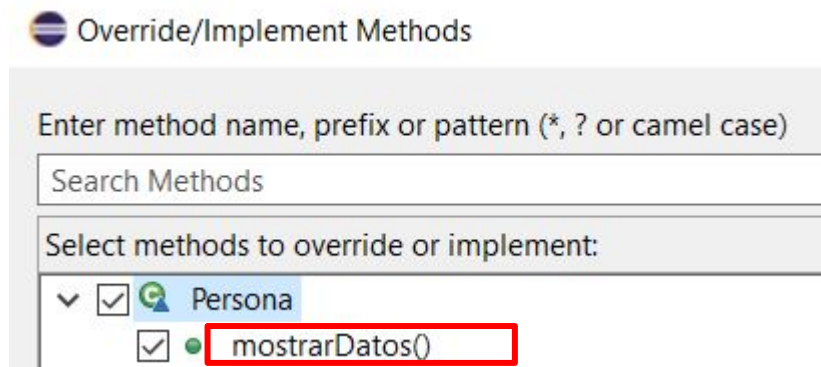
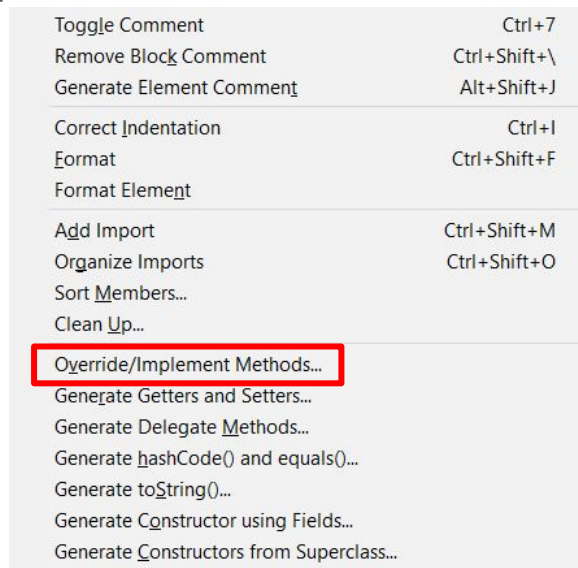
- Una subclase puede **redefinir, sobrescribir, modificar** un método de los que ha heredado de su superclase
- La palabra que se utiliza en inglés es **override**
- Cuando sabemos que vamos a sobrescribir un método en una subclase, podemos ayudarnos de Eclipse pulsando botón derecho->Source sobre el código



Toggle Comment	Ctrl+7
Remove Block Comment	Ctrl+Shift+\
Generate Element Comment	Alt+Shift+J
Correct Indentation	Ctrl+I
Format	Ctrl+Shift+F
Format Element	
Add Import	Ctrl+Shift+M
Organize Imports	Ctrl+Shift+O
Sort Members...	
Clean Up...	
Override/Implement Methods...	
Generate Getters and Setters...	
Generate Delegate Methods...	
Generate hashCode() and equals()...	
Generate toString()...	

Herencia de métodos. Sobrescritura

- En el ejercicio que estamos haciendo tenemos los métodos `mostrarDatosPersona`, `mostrarDatosAlumno`, `mostrarDatosProfesor`
- Pero lo más óptimo sería declarar el método `mostrarDatos` en la superclase y que las subclases lo **sobrescriban para adaptarlo** a sus necesidades



Herencia de métodos. Sobrescritura

- Al sobrescribir un método con el asistente aparece este código:

```
@Override
public String mostrarDatos() {
    // TODO Auto-generated method stub
    return super.mostrarDatos();
}
```

- `@Override` indica que es un método sobrescrito
- `super.mostrarDatos()` accede al método `mostrarDatos` de la clase padre

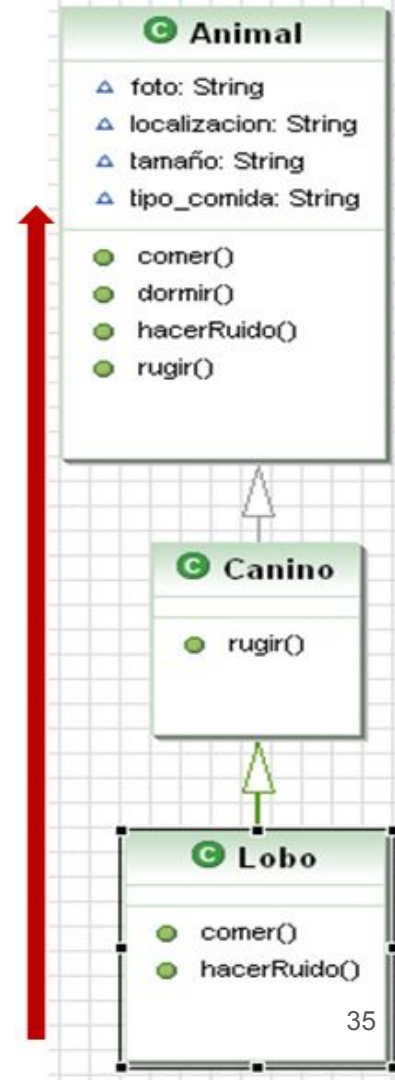
Herencia de métodos. Sobrescritura

- **super** permite acceder a los métodos de la superclase
- En un método sobrescrito permite modificar/ampliar el método de la superclase
- Por ejemplo, al sobrescribir el método `mostrarDatos` en la clase `Alumno`:

```
@Override
public String mostrarDatos() {
    //Devuelve la información de mostrarDatos de Persona
    //y le concatena la propia del Alumno (grupo y nota)
    return super.mostrarDatos()+" Grupo: "+grupo+
        " Nota Media: "+notaMedia;
}
```

Herencia de métodos. Sobrescritura

- Al ejecutar un método sobrescrito, se **busca** el método en esa clase
- Si no existe, se busca **sucesivamente en las superclases**
- lobo.comer() ejecuta el método sobrescrito de la clase Lobo
- lobo.rugir() ejecuta el método sobrescrito de la clase Canino



Herencia de métodos. Sobrescritura

EJERCICIO

- Modifica el ejercicio para que la clase Persona implemente el método mostrarDatos y el resto de clases lo sobrescriban
- Reutiliza código utilizando la sentencia super
- Elimina los métodos mostrarDatosXXX de cada una de las clases
- Comprueba el funcionamiento del método mostrarDatos en cada objeto creado

```
@Override
public String mostrarDatos() {
    return super.mostrarDatos()+" "+grupo+" "+notaMedia;
}
```

```
@Override
public String mostrarDatos() {
    return super.mostrarDatos()+" "+especialidad+" "+salario;
}
```

```
System.out.println("Datos persona 1: " + persona1.mostrarDatos());
System.out.println("Datos persona 2: " + persona2.mostrarDatos() + "\n");

System.out.println("Datos alumno 1: " + alumno1.mostrarDatos());
System.out.println("Datos alumno 2: " + alumno2.mostrarDatos() + "\n");

System.out.println("Datos profesor 1: " + profesor1.mostrarDatos());
System.out.println("Datos profesor 2: " + profesor2.mostrarDatos() + "\n");
```

Herencia de métodos. Sobrescritura. toString()

- En realidad, el método mostrarDatos que estamos sobrescribiendo de la clase Persona, es el método toString que ya conocemos
- toString es un método de la clase Object
- Todas las clases heredan de Object
- Y por tanto pueden sobrescribir ese método

Herencia de métodos. Sobrescritura. toString()

EJERCICIO

- Modifica el ejercicio para que la clase Persona sobrescriba el método toString y el resto de clases lo sobrescriban
- Copia el código de los métodos mostrarDatos. Vuelve a utilizar super para ahorrar código
- Elimina los métodos mostrarDatos (ya no son necesarios) de Persona, Alumno y Profesor

```
@Override
public String toString() {
    return nombre+" "+apellidos+" "+fechaNacimiento;
}
```

```
@Override
public String toString() {
    return super.toString()+" "+grupo+" "+notaMedia;
}
```

```
@Override
public String toString() {
    return super.toString()+" "+especialidad+" "+salario;
}
```

```
System.out.println("Datos persona 1: " + persona1.toString());
System.out.println("Datos persona 2: " + persona2.toString() + "\n");
```

```
System.out.println("Datos alumno 1: " + alumno1.toString());
System.out.println("Datos alumno 2: " + alumno2.toString() + "\n");
```

```
System.out.println("Datos profesor 1: " + profesor1.toString());
System.out.println("Datos profesor 2: " + profesor2.toString() + "\n");
```


Herencia de métodos. Sobrescritura

CONCLUSIÓN

- Las subclases **heredan todos los métodos** de su superclase
- Las subclases **pueden NO modificar** los métodos heredados
 - Ejemplos: métodos get y set heredados
- Las subclases pueden modificar (**sobrescribir**) los métodos heredados para adaptar su funcionamiento
 - Ejemplos: toString en Persona muestra los datos de Persona y toString de Alumno muestra los datos de Persona+Alumno

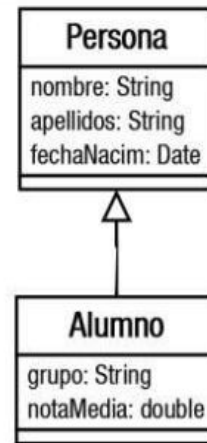
Instanciar objetos heredados

- Java permite declarar objetos de una clase padre e instanciarlos como una clase hijo

```
Persona alumno1;  
alumno1=new Alumno();
```

- alumno1 está **declarado** como Persona
- pero **instanciado (creado)** como Alumno
- alumno1 solo podrá utilizar los **métodos**:
 - de la clase con la que se **declara** (Persona)
 - de la clase Alumno que **sobrescriban** métodos de la clase Persona

CLASE ALUMNO



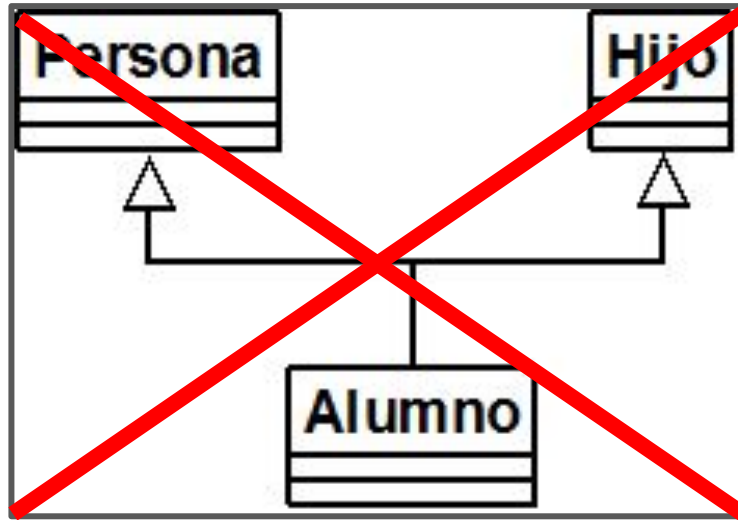
Instanciar objetos heredados

EJERCICIO

- En nuestro proyecto, ¿qué métodos puede usar alumno1? Compruébalo en Eclipse
- Declara un objeto Persona de nombre profesor1 e instáncialo como Profesor
- Muestra los datos del alumno y del profesor con el método toString()
- ¿Qué método se ha ejecutado en cada caso?

Herencia múltiple

- En java no existe la posibilidad de que un objeto herede de dos clases



Uso de la herencia

La herencia en POO se puede implementar como

- Una **especialización**
 - Las superclases son más generales y las subclases más específicas
 - Alumno es una especialización de Persona
 - Van de más general a más específico: Animal->Mamífero->Canino->Lobo
- Una **generalización**
 - Se agrupan clases que tienen características comunes
 - Cuadrado, círculo, triángulo pueden calcular su área y perímetro, luego tiene sentido poner esa funcionalidad en una clase genérica como Figura

Uso de la herencia

En clases que nosotros creamos:

- No debemos usar la herencia “porque sí”
 - Reutilizar código no es una justificación válida de su uso
- Podemos usar la herencia cuando se cumpla “**Es un...**”
 - Alumno **es una** Persona
 - Círculo **es una** Figura
 - Lobo **es un** Canino y Canino **es un** tipo de Animal

Uso de la herencia

- Pero muchas veces utilizaremos la herencia con **clases que ya existen** en las librerías java
- Ejemplo de herencia en la clase FileReader que utilizamos para leer ficheros de texto:

java.lang.Object

java.io.Reader

java.io.InputStreamReader

java.io.FileReader

- Hereda de la clase InputStreamReader
- Que a su vez hereda de Reader
- Y esta de Object (como todas las Clases)

Uso de la herencia

EJERCICIO

- Accede a la API de java y revisa la documentación relativa a la clase `FileReader` e indica:
 - Sus constructores
 - Sus atributos y métodos propios
 - Sus atributos y métodos heredados de otras clases

Herencia

EJERCICIOS

Hacer las actividades de Herencia del Campus Virtual

Herencia

[Enlace con los ficheros .java](#)

FIN!